



Práctica 04: Flujo Completo Request-Response



Objetivos

- Implementar un flujo end-to-end completo
- Ver todas las capas trabajando juntas
- Entender las transformaciones en cada paso
- Crear una operación de negocio compleja



Descripción

En esta práctica implementarás el flujo completo de **crear un pedido**, que involucra múltiples operaciones:

1. Validar usuario existe
2. Validar productos y stock
3. Calcular totales (subtotal, impuestos, envío)
4. Crear el pedido con sus items
5. Reducir stock de productos
6. Todo en una transacción



Estructura

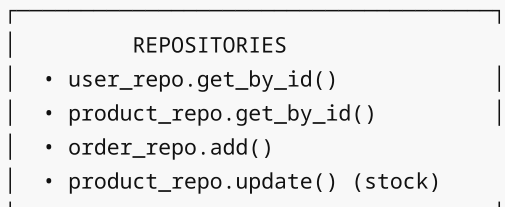
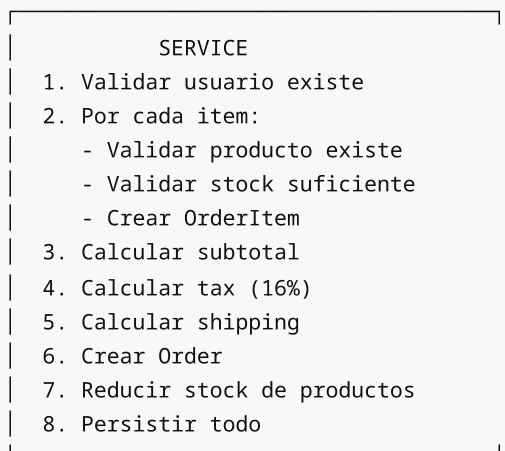
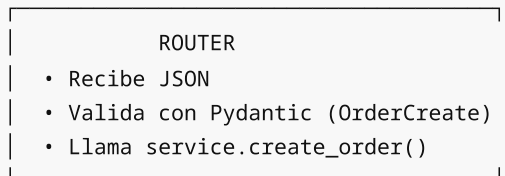
```
starter/  
├─ main.py  
├─ database.py  
├─ models/  
│   ├─ user.py  
│   ├─ product.py  
│   └─ order.py          # Order + OrderItem  
├─ schemas/  
│   ├─ user.py  
│   ├─ product.py  
│   └─ order.py          # OrderCreate, OrderResponse  
├─ repositories/  
│   ├─ user.py  
│   ├─ product.py  
│   └─ order.py  
├─ services/  
│   └─ order.py          # Lógica compleja  
├─ exceptions/  
│   ├─ base.py  
│   ├─ user.py  
│   ├─ product.py  
│   └─ order.py  
├─ handlers/  
│   └─ exception_handlers.py  
└─ routers/  
    └─ orders.py
```



Flujo de Crear Pedido

```
POST /orders/  
{
```

```
"user_id": 1,
"items": [
  {"product_id": 5, "quantity": 2},
  {"product_id": 8, "quantity": 1}
],
"shipping_address": "123 Main St"
}
```



DATABASE

Pasos

Paso 1: Modelos

Abre `starter/models/` y descomenta:

- `User` - usuario básico
- `Product` - producto con stock
- `Order` y `OrderItem` - pedido con relación

Paso 2: Schemas

Abre `starter/schemas/order.py` y descomenta:

- `OrderItemCreate` - item de entrada
- `OrderCreate` - pedido de entrada
- `OrderItemResponse` - item de salida
- `OrderResponse` - pedido de salida

Paso 3: Excepciones

Abre `starter/exceptions/` y descomenta todas las excepciones.

Paso 4: Repositories

Abre `starter/repositories/` y descomenta los repositories.

Paso 5: Order Service

Abre `starter/services/order.py` - **ESTE ES EL CORAZÓN**.

Descomenta y estudia `create_order()` :

- Cómo valida el usuario
- Cómo itera sobre items
- Cómo calcula totales
- Cómo maneja la transacción

Paso 6: Router

Abre `starter/routers/orders.py` y descomenta el endpoint.

Paso 7: Main

Descomenta todo en `main.py` .

✓ Verificación

```
cd starter
uvicorn main:app --reload
```

Crea datos de prueba primero (o usa el script `/seed`):

```
# Crear usuario
POST /users/
{"name": "John", "email": "john@example.com"}

# Crear productos
POST /products/
{"name": "Widget", "sku": "WDG-001", "price": 29.99, "stock": 10}

POST /products/
{"name": "Gadget", "sku": "GDG-001", "price": 49.99, "stock": 5}
```

Luego crea el pedido:

```
POST /orders/
{
  "user_id": 1,
  "items": [
```

```
{
  "product_id": 1, "quantity": 2},
  {"product_id": 2, "quantity": 1}
],
"shipping_address": "123 Main St, City"
}
```

Response esperada:

```
{
  "id": 1,
  "user_id": 1,
  "status": "pending",
  "items": [
    {
      "id": 1,
      "product_id": 1,
      "product_name": "Widget",
      "quantity": 2,
      "unit_price": 29.99,
      "subtotal": 59.98
    },
    {
      "id": 2,
      "product_id": 2,
      "product_name": "Gadget",
      "quantity": 1,
      "unit_price": 49.99,
      "subtotal": 49.99
    }
  ],
  "subtotal": 109.97,
  "tax": 17.60,
  "shipping_cost": 0,
  "total": 127.57,
  "created_at": "2025-01-01T12:00:00Z"
}
```

Puntos Clave

- **Service orquesta:** Coordina múltiples repositories
- **Transacción implícita:** Todo en una sesión
- **Validaciones de negocio:** En el service, no en el router
- **DTOs separados:** OrderCreate ≠ OrderResponse
- **Excepciones específicas:** UserNotFoundError, InsufficientStockError